

Machine Learning A (2026)

Home Assignment 5

Ludovico Maria Spitaleri, DMH249

Contents

1 On the Role of Dependence (8 points) [Sadegh, W5 Monday]	1
2 On Confidence Intervals (12 points) [Sadegh, W5 Monday]	1
2.1 Part 1	1
2.2 Part 2	1
3 Loss Range Correction in Generalization Bounds (10 points) [Sadegh, W5 Monday]	1
3.1 (i)	1
3.2 (ii)	1
4 Convolutional Neural Networks (70 points) [Christian, W5 Friday]	1
4.1 Sobel filter (24 points) [Christian, W5 Friday]	1
4.2 Convolutional neural networks (24 points) [Christian, W5 Friday]	2
4.3 Augmentation (22 points) [Christian, W5 Friday]	3

1 On the Role of Dependence (8 points) [Sadegh, W5 Monday]

2 On Confidence Intervals (12 points) [Sadegh, W5 Monday]

2.1 Part 1

2.2 Part 2

3 Loss Range Correction in Generalization Bounds (10 points) [Sadegh, W5 Monday]

3.1 (i)

3.2 (ii)

4 Convolutional Neural Networks (70 points) [Christian, W5 Friday]

4.1 Sobel filter (24 points) [Christian, W5 Friday]

The Sobel filter is implemented with a single `nn.Conv2d` with one input and two output channels. The magnitude G is obtained by squaring, summing over the channel axis and taking the square root.

```

Kx = torch.tensor([[1., 0., -1.],
                   [2., 0., -2.],
                   [1., 0., -1.]])
Ky = torch.tensor([[1., 2., 1.],
                   [0., 0., 0.],
                   [-1., -2., -1.]])
K = torch.stack([Kx, Ky]).unsqueeze(1)
K = torch.flip(K, dims=[-2, -1])

conv = nn.Conv2d(1, 2, kernel_size=3, padding=1, bias=False)
conv.weight = nn.Parameter(K, requires_grad=False)

g = conv(x)
G = torch.sqrt(torch.sum(torch.square(g), dim=1))
Gx, Gy = g[0, 0], g[0, 1]

```

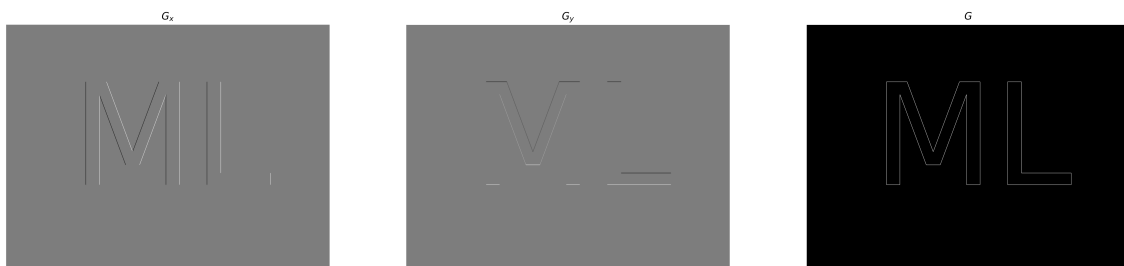


Figure 1: The Sobel feature maps G_x , G_y and G .

`nn.Conv2d` does not flip, it computes the cross-correlation $[K \star I]_{ij} = \sum_{u,v} K_{uv} I_{i+u, j+v}$. Using G_x, G_y unchanged would therefore compute the correlation with those matrices, which equals the convolution with the *flipped* matrices, so G_x and G_y would come out with the opposite sign. To implement the convolution the kernels must be flipped before assigning them as weights.

The final map G is unaffected by this choice, since it only depends on G_x^2 and G_y^2 and a flip of these antisymmetric kernels only changes the sign of G_x and G_y .

4.2 Convolutional neural networks (24 points) [Christian, W5 Friday]

```

class Net(nn.Module):
    def __init__(
        self,
        img_size=28,
        in_channels=3,  # r, g, b
        n_maps=64,
        kernel_size=5,
        pool_size=2,
        n_classes=43,
    ):
        super(Net, self).__init__()

```

```

        self.conv1 = nn.Conv2d(in_channels, n_maps,
kernel_size=kernel_size)
        size = img_size - kernel_size + 1
        self.pool1 = nn.MaxPool2d(pool_size)
        size = size // pool_size
        self.conv2 = nn.Conv2d(n_maps, n_maps, kernel_size=kernel_size)
        size = size - kernel_size + 1
        self.pool2 = nn.MaxPool2d(pool_size)
        size = size // pool_size

        self.fc2 = nn.Linear(n_maps * size * size, n_classes)

    def forward(self, x):
        x = self.pool1(F.elu(self.conv1(x)))
        x = self.pool2(F.elu(self.conv2(x)))
        x = torch.flatten(x, 1)
        return self.fc2(x)

```

Printing the model gives the expected output:

```

Net(
  (conv1): Conv2d(3, 64, kernel_size=(5, 5), stride=(1, 1))
  (pool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
  (conv2): Conv2d(64, 64, kernel_size=(5, 5), stride=(1, 1))
  (pool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
  (fc2): Linear(in_features=1024, out_features=43, bias=True)
)

```

4.3 Augmentation (22 points) [Christian, W5 Friday]

During training, each image is

1. resized to 32×32
2. randomly rotated by an angle between $[-5^\circ, 5^\circ]$
3. randomly cropped to 28×28
4. colour-jittered with a brightness factor between $[0.2, 1.8]$ and contrast factor between $[0.6, 1.4]$
5. if it's part of the classes 11, 12, 13, 17, 18, 26, 30, 35, the image is horizontally flipped with probability 0.5.

If not training, all images are just cropped to 28×28 in the center to have a consistent size with the training images.

Since augmentation must not change the label, only signs of classes 11, 12, 13, 17, 18, 26, 30 and 35 are horizontally flipped. This is because they are almost symmetric, so a flipped image still shows the same class. For other classes a flip changes the meaning: a flipped arrow

becomes the opposite direction, or a speed limit could have a wrong number on it after the transformation.

An extra useful transformations that could be added in the training is a change in the perspective:

```
image = transforms.RandomPerspective(distortion_scale=0.15, p=0.5)
(image)
```

This would help the model train on recognizing signs even when not show on a perfect angle. This is a very common situation in real-world driving when encountering signs, especially if on the side of tight curves.